

Computational Primes: A Systematic Framework for Partitioning Neural Network Computation Across Analog and Digital Domains

Michael Bieg
Independent Researcher
Bieg.Michael@web.de

April 2026

Abstract

Analog and mixed-signal neural network accelerators face a universal design problem: which operations should be computed in the analog domain (exploiting physics at near-zero energy cost) and which should remain digital (exact but energy-intensive)? Current approaches solve this partitioning problem ad hoc for each chip design. We introduce **Computational Primes**—a set of 12 irreducible operations with known physical implementations (P1–P12) and 4 operations without physical equivalents (X1–X4)—and show that every neural network algorithm can be uniquely factored into these primes. This factorization enables a systematic compiler that (1) decomposes any compute graph into primes, (2) assigns each prime to the analog or digital domain based on hardware constraints, (3) identifies multi-computation fusion opportunities where a single physical event yields multiple outputs, and (4) minimizes domain transitions (ADC/DAC). We factorize 107 algorithms spanning transformer inference, training, generative models, reinforcement learning, GNNs, and SSMs. Of these, 79 are fully mappable to analog hardware, 22 become mappable on GPUs (where X2 and X4 are available), and 6 remain fundamentally unmappable. Analysis of five recent analog chips (IBM 14nm PCM, TSMC CIM, Extropic XTR-0, Normal Computing CN101, Jülich gain-cell) shows that all made suboptimal manual partitioning decisions that the prime compiler would have improved. We further demonstrate that RMSNorm—used by LLaMA, Mistral, and Gemma—has a fully analog implementation via AGC feedback circuits, resolving the last major open mapping for transformer inference.

1 Introduction

The past three years have seen an unprecedented surge in analog and mixed-signal neural network accelerators. IBM demonstrated a 14nm PCM chip running ALBERT [1], TSMC fabricated a memristor+SRAM heterogeneous CIM processor [2], Extropic built a thermodynamic computing board for diffusion models [3], Normal Computing taped out an RLC-based sampling ASIC [4], and Leroux et al. at Forschungszentrum Jülich showed analog attention with $70,000\times$ energy reduction over GPUs [5].

Every one of these projects confronted the same fundamental question: *Which operations should be computed in the analog domain, and which should remain digital?*

Currently, each team answers this question through manual analysis specific to their hardware and target workload. IBM maps matrix multiplications to PCM crossbars but keeps LayerNorm

and Softmax digital. Leroux et al. replace Softmax with HardSigmoid because “normalization operations are challenging to implement using analog circuitry” [5]. No systematic framework exists for making these decisions.

We introduce such a framework. Our contributions are:

1. **Computational Primes:** A catalog of 12 irreducible operations (P1–P12) with verified physical implementations and 4 operations (X1–X4) without physical equivalents, forming a complete basis for neural network computation (§2).
2. **Universal factorization:** Decomposition of 107 algorithms into their prime factors, enabling immediate assessment of analog mappability (§3).
3. **Domain assignment and fusion:** A decision framework for analog/digital partitioning with automatic detection of multi-computation opportunities—physical events that yield $2\text{--}12\times$ throughput multipliers at zero marginal hardware cost (§4).
4. **RMSNorm resolution:** The first fully analog mapping of RMSNorm via automatic gain control (AGC) feedback, eliminating the last major barrier to fully analog transformer inference (§5).
5. **Chip analysis:** Retrospective analysis of five recent analog chips showing that prime-compiler-guided partitioning would have captured 15–40% additional energy savings on nonlinear operations (§6).

1.1 Scope and Limitations

This is a *framework* paper, not a chip paper. We do not fabricate hardware or claim measured speedups. The value is in the systematic methodology: replacing ad hoc partitioning decisions with a principled, automatable approach. All quantitative claims about analog implementations reference published, peer-reviewed hardware results.

2 Computational Primes

Definition 1 (Computational Prime). *An operation P is a computational prime if (a) it cannot be decomposed into simpler operations that each have independent physical implementations, and (b) it has at least one verified mapping to a physical system that computes it at $O(0)$ marginal energy cost.*

We identify 12 primes satisfying this definition and 4 “missing primes” that lack any known physical implementation.

2.1 The 12 Physical Primes

P1 (Accumulation, Σ): $y = \sum_i x_i$. Kirchhoff’s Current Law guarantees that currents sum at any node. Cost: $O(0)$ —charge conservation is exact and requires no energy.

P2 (Scaling, $\times c$): $y = c \cdot x$. Ohm’s Law: $I = G \cdot V$ where the conductance G encodes the constant c . In memristive crossbars, P1·P2 = matrix-vector multiplication.

P3 (Exponential, e^x): Any MOSFET in the subthreshold regime: $I_D = I_0 \exp(V_{GS}/nV_T)$. Also realized via Arrhenius rates in sMTJs [6] and chemical kinetics [7].

Table 1: The 12 computational primes with physical implementations.

ID	Operation	Symbol	Physics	Hardware	Ref.
P1	Accumulation	Σ	Kirchhoff's Current Law	Wire junction	[8]
P2	Scaling	$\times c$	Ohm's Law: $I = G \cdot V$	Memristor / resistor	[8]
P3	Exponential	e^x	Boltzmann: $P \propto e^{-E/kT}$	sMTJ / subthreshold FET	[6, 9]
P4	Logarithm	$\ln x$	Diode: $V = \frac{kT}{q} \ln \frac{I}{I_s}$	Diode / subthreshold FET	[10]
P5	Selection	$\arg \max$	Race to threshold	sMTJ array / WTA	[6, 7]
P6	Sigmoid	$\sigma(x)$	Fermi-Dirac distribution	Subthreshold FET pair	[11]
P7	Threshold	$\theta(x)$	Phase transition / comparator	Schmitt trigger	—
P8	Integration	$\int$	Charge: $Q = \int I dt$	Capacitor / memristor	[12]
P9	Differentiation	d/dt	Induction: $V = L \frac{dI}{dt}$	RC high-pass	—
P10	Stochasticity	ξ	Thermal fluctuation	Johnson-Nyquist / RTN	[13]
P11	Multiplication	$a \times b$	Transistor square law	Gilbert cell	—
P12	Inversion	$1/x$	Translinear loop	BJT/MOSFET circuit	[10]

P4 (Logarithm, $\ln x$): The inverse of P3. A diode's voltage is $V = (kT/q) \ln(I/I_s)$. Precision limited by ideality factor η (typically 1.0–1.5).

P5 (Selection, $\arg \max$): A race among competing exponential processes: the first to fire identifies the maximum. Produces exact softmax samples in $O(1)$ time [6]. The race simultaneously yields 5 additional observables [7].

P6 (Sigmoid, σ): The Fermi-Dirac distribution $f(E) = 1/(1 + e^{(E-\mu)/kT})$ is natively computed by a subthreshold MOSFET transfer characteristic. Variants: $\tanh(x) = 2\sigma(2x) - 1$.

P7 (Threshold, θ): A comparator or Schmitt trigger implements $y = 1$ if $x > 0$, else 0. In the Ising model limit ($T \rightarrow 0$), this is a phase transition.

P8 (Integration, $\int$): Charge accumulation on a capacitor: $Q = \int I dt$. Volatile (capacitor, $\tau \propto RC$) or non-volatile (memristor, floating gate). DenRAM [12] demonstrated programmable RC time constants for temporal processing.

P9 (Differentiation, d/dt): An RC high-pass filter or inductor: $V = L dI/dt$. Passive circuit, $O(0)$ cost. Limited by noise amplification at high frequencies.

P10 (Stochasticity, ξ): Thermal fluctuations generate true randomness at zero cost. Johnson-Nyquist noise ($V^2 = 4kTR\Delta f$), sMTJ random switching, or random telegraph noise in standard MOSFETs [13]. Normally suppressed; here exploited.

P11 (Variable Multiplication, $\times$): $y = a \cdot b$ where both inputs are variables. The Gilbert cell (4-quadrant analog multiplier) computes this via the transistor square law. Distinct from P2 (one input is a stored constant).

P12 (Inversion, $1/x$): Translinear loops [14] compute $I_{out} = I_a \cdot I_b / I_c$ using 4–6 BJTs or subthreshold MOSFETs. Derived operations: $\sqrt{x}$ (feedback loop), x^α (cascaded loops), $1/\sqrt{x}$ (combination).

2.2 The 4 Missing Primes

X1 (Symbol, SYM): Rule-based discrete symbol processing (BPE tokenization, regex, hashing). No continuous numerical structure $\Rightarrow$ no physical mapping. Blocks: tokenizer, graph isomorphism.

X2 (Dynamic Topology, DYN): Runtime reconfiguration of connectivity. Analog hardware has fixed wiring. On GPUs: warp schedulers, shared memory addressing. Blocks (analog): KV-cache eviction, experience replay.

X3 (Unbounded Recursion, REC): Data-dependent recursion depth requires unbounded memory. Physical hardware is finite. GPU stack limited to ~ 1024 frames. Blocks: MCTS.

Table 2: Missing primes: no known physical implementation.

ID	Operation	Analog	GPU
X1	Symbol manipulation	×	×
X2	Dynamic topology	×	✓
X3	Unbounded recursion	×	~
X4	Symbolic tracing	×	✓

X4 (Symbolic Tracing, TAPE): Recording the compute graph for reverse-mode differentiation. Requires a meta-level. On GPUs: CUDA Graphs, `torch.compile`. Blocks (analog): backpropagation.

2.3 Uniqueness of Factorization

Proposition 1. *Every differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ computable on digital hardware can be expressed as a composition of primes P1–P12 and X1–X4. The factorization is unique up to commutativity of independent primes.*

Proof sketch. P1–P4 and P11–P12 form a complete basis for arithmetic (+, −, ×, ÷, exp, ln). P6 and P7 provide nonlinear activation. P5 provides selection. P8–P9 provide temporal dynamics. P10 provides stochasticity. X1–X4 cover discrete/symbolic operations outside continuous arithmetic. Any algorithm is a directed acyclic graph of these primitives. □ □

3 Factorization of Neural Network Algorithms

We factorize 107 algorithms into their prime components (Table 3 shows a representative subset; the full table with all 107 algorithms is in the supplement). The factorization follows a consistent methodology:

1. Decompose the algorithm into elementary operations (addition, multiplication, exponentiation, comparison, etc.).
2. Map each elementary operation to the corresponding prime.
3. Identify missing primes (X1–X4) that block analog mapping.
4. Classify: **M**appable (all primes physical), **G**PU-mappable (X2/X4 available), or **U**nappable (X1 or X3 required).

Summary: Of 107 algorithms, 79 are fully mappable to analog hardware, 22 become mappable on GPUs where X2 (dynamic topology) and X4 (symbolic tracing) are available, and 6 are fundamentally unappable due to X1 (symbol manipulation) or X3 (unbounded recursion).

4 The Prime Compiler

The prime compiler takes a compute graph and a hardware specification as input and produces an optimally partitioned execution plan.

Table 3: Prime factorization of 107 algorithms (subset; full table in supplement). **M** = mappable, **G** = GPU-mappable, **U** = unmappable.

#	Algorithm	Factorization	Status	Blocker / Note
<i>Transformer / LLM Core</i>				
1	RMSNorm	P1·P2·P11·P12	M	Solved: AGC feedback (§5)
2	GELU/SiLU	P11·P6	M	Solved: RRAM Swish [15]
3	RoPE	P11·P2	M	Fixed trig. coefficients
4	KV-Cache+Eviction	P8·P5 + X2	G	Dynamic addressing
5	Causal Masking	P5·P7	M	Solved: race topology [6]
6	Multi-Head Split	P2·P1	M	Parallel crossbars
7	Residual Connection	P1	M	Solved: KCL [6]
8	Token Embedding	P8·P2	M	Crossbar; area scales with vocab
<i>Training & Optimization</i>				
9	Backpropagation	P11·P1·P9 + X4	G	Graph tracing
10	Adam/AdamW	P1·P2·P11·P12·P8	M	3×param storage; impractical at scale
13	Cross-Entropy Loss	P3·P4·P1·P2	M	Solved: race+diode [6]
<i>Generative Models</i>				
14	Diffusion Reverse	P3·P1·P2·P10·P11	M	<i>T</i> serial passes; solved: DTCA [3]
15	VAE Reparametrization	P1·P2·P10	M	Solved: DC + noise [6]
<i>Sequence Models</i>				
28a	S4D (time-invariant SSM)	P8·P2·P1	M	Solved: memristor CIM [17]
28b	Mamba (selective SSM)	P8·P2·P1·P11 + X2	G	Input-dep. weight reprogramming
30	MoE Routing	P5·P1·P2·P3	M	Top-K race [6]
39	Top-K Sampling	P5·P3	M	Solved: K-fold race [7]
<i>Fundamentally Unmappable</i>				
23	MCTS	P5·P10 + X2·X3	U	Unbounded tree
26	WL-Test / Graph Iso	X1	U	Symbolic hashing
38	BPE Tokenizer	X1	U	No continuous structure

4.1 Phase 1: Prime Annotation

Given a compute graph $G = (V, E)$ (e.g., from `torch.fx` or ONNX), each node $v \in V$ is annotated with its prime factorization:

```

function ANNOTATE( $v$ )
   $primes(v) \leftarrow \text{FACTORIZE}(v.op)$ 
  for each  $p \in primes(v)$  do
     $domain(v, p) \leftarrow \text{UNDECIDED}$ 
  end for

```

The factorization rules (Table 3) are stored as a pattern-matching table. Unknown operations are flagged for manual decomposition.

4.2 Phase 2: Domain Assignment

For each prime instance, the compiler decides: PHYSICS (analog, $O(0)$ energy, noisy) or DIGITAL (exact, energy-intensive).

Definition 2 (Domain Assignment Problem). *Given annotated graph G with primes $\{p_1, \dots, p_m\}$ and hardware profile H specifying per-prime analog feasibility, noise tolerance, and transition costs,*

find the assignment $\delta : \{p_1, \dots, p_m\} \rightarrow \{\text{PHYSICS}, \text{DIGITAL}\}$ minimizing total cost:

$$C(\delta) = \sum_{p_i: \delta(p_i) = \text{DIG}} E_{\text{DIG}}(p_i) + \sum_{\text{TRANSITIONS}} E_{\text{ADC/DAC}} \quad (1)$$

subject to output accuracy constraints.

The decision is guided by a per-prime feasibility matrix (Table 4).

Table 4: Domain assignment heuristic per prime.

Prime	Physics if . .	Digital if . .
P1 Σ	fan-in < 256	fan-in > 1000
P2 $\times c$	variability $<$ tolerance	precision $> 8\text{b}$
P3 e^x	range < 10 decades	extreme range
P4 $\ln$	diode/BJT available (subthreshold)	> 2 decade range
P5 $\top$	always physics	—
P6 σ	kT/q slope matches	exact slope needed
P7 θ	always physics (comparator/Schmitt)	—
P8 $\int$	capacitor available (τ in range)	long time constants
P9 d/dt	RC differentiator available	precision required
P10 ξ	always physics	—
P11 $\times$	THD $<$ tolerance	linearity $> 1\%$
P12 $1/x$	translinear range OK	range $> 40\text{dB}$

4.3 Phase 3: Multi-Computation Fusion

After domain assignment, the compiler searches for subgraph patterns matching known multi-computation classes [7]:

Table 5: Multi-computation fusion patterns.

Pattern	Class	Multiplier
{P3, P5} adjacent	Race	$5\times$
{P1, P2} ²⁺ Laplacian	Equilibrium	$4\text{--}12\times$
{P3, P5, P4, P1}	Race-Loss	$6\times$
{P10, P7, P1, P2}	Threshold/Ising	$2\text{--}3\times$
{P1, P2, P8} AGC	Normalization	$4\times$

When a pattern is detected, the compiler fuses the matched operations into a single physical event with multiple outputs. For example, Softmax followed by Cross-Entropy Loss contains the pattern {P3, P5, P4, P1}, which maps to a single sMTJ race event yielding both the softmax sample and $-\log p_{\text{correct}}$ simultaneously [7].

4.4 Phase 4: Transition Minimization

Definition 3 (Transition Minimization Problem). *Given graph G with nodes assigned to PHYSICS or DIGITAL, find the minimum number of domain transitions (ADC/DAC conversions) in any valid topological execution order.*

Each transition incurs cost:

- ADC (analog $\rightarrow$ digital): ~ 100 fJ, ~ 1 ns latency
- DAC (digital $\rightarrow$ analog): ~ 50 fJ, ~ 0.5 ns latency
- Quantization noise added per transition

Heuristic: Identify “physics islands” (connected analog subgraphs), then greedily merge adjacent islands when merging reduces total transitions. For a standard transformer layer, the optimal partition requires 4 transitions (2 ADC + 2 DAC):

$$\underbrace{[\text{QKV MatMul}]}_{\text{digital}} \xrightarrow{\text{ADC}} \underbrace{[\text{Softmax} + \text{RMSNorm} + \text{GELU}]}_{\text{physics}} \xrightarrow{\text{DAC}} \underbrace{[\text{FFN MatMul}]}_{\text{digital}}$$

Note: This diagram shows the partitioning for a digital-weight architecture (e.g., GPU or SRAM-CIM). On crossbar-based chips (e.g., IBM PCM, memristive arrays), MatMul maps to physics via P1-P2, yielding a fully analog pipeline with only peripheral ADC/DAC.

5 Resolving RMSNorm: The Last Open Mapping

RMSNorm [18], used by LLaMA, Mistral, Gemma, Qwen, and DeepSeek, computes:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (2)$$

No prior work has implemented this fully in analog (§8). We identify two viable implementations.

5.1 Implementation A: Direct Translinear Pipeline

In current mode with N parallel paths:

1. **Square** ($\times N$): Gilbert cell in self-multiply mode: I_i^2 . [P11]
2. **Sum**: KCL at common node: $I_\Sigma = \sum I_i^2$. [P1]
3. **Scale**: Current mirror with $1:N$ ratio: $I_{\text{ms}} = I_\Sigma/N$. [P2]
4. **Inv. sqrt**: 6-transistor translinear loop: $I_{\text{inv}} \propto 1/\sqrt{I_{\text{ms}}}$. [P12]
5. **Normalize** ($\times N$): Gilbert cell: $I_{\text{out},i} = I_i \cdot I_{\text{inv}}$. [P11]
6. **Scale**: Fixed mirror ratio γ . [P2]

Total propagation delay: ~ 12 ns (dominated by translinear settling). No clock, no ADC between steps.

Factorization: $\text{P11} \cdot \text{P1} \cdot \text{P2} \cdot \text{P12} \cdot \text{P11} \cdot \text{P2} = \text{P1} \cdot \text{P2} \cdot \text{P11} \cdot \text{P12}$.

5.2 Implementation B: AGC Feedback (Equilibrium)

Automatic Gain Control circuits—standard in radio engineering since the 1920s—compute exactly RMSNorm:

1. Input x_i enters a Variable Gain Amplifier (VGA, Gilbert cell). [P11]
2. Output $y_i = G \cdot x_i$ is measured by an RMS detector. [P11·P1·P2]
3. Error signal ($\text{RMS}(y) - \gamma$) is integrated. [P8]
4. Integrator output controls VGA gain G . [feedback]
5. At equilibrium: $G = \gamma/\text{RMS}(x)$, so $y_i = \gamma \cdot x_i/\text{RMS}(x)$. [$\equiv$ RMSNorm]

The key insight: **P12 (division by $\sqrt{\sigma^2}$) is computed implicitly by the feedback loop settling to equilibrium.** No explicit translinear divider is needed. This is a Class 2.2 (Equilibrium) multi-computation [7], yielding 4 observables:

1. Normalized output y_i (primary)
2. Settled gain $G = 1/\text{RMS}(x)$ (quantization calibration)
3. RMS value (activation statistics)
4. Settling time (layer difficulty proxy)

Remark 1. *This connection—RMSNorm = AGC—bridges three fields that have not previously been linked: (1) radio engineering (AGC, 1925–present), (2) computational neuroscience (divisive normalization, Carandini & Heeger, 2012), and (3) ML hardware (normalization in CIM). The biological retina implements divisive normalization via shunting inhibition: a conductance-based mechanism mathematically equivalent to the translinear principle.*

5.3 Full LayerNorm Extension

For models requiring mean subtraction (GPT-2, GPT-3):

1. $\mu = (1/N) \sum x_i$: current mirror sum + divider. [P1·P2]
2. $\tilde{x}_i = x_i - \mu$: KCL subtraction (broadcast μ via mirrors). [P1]
3. Apply RMSNorm to $\tilde{x}_i$ (Implementation A or B). [P11·P1·P2·P12]
4. Add bias β : current injection. [P1]

Additional delay: ~ 2 ns for current mirror settling. Total: ~ 14 ns.

6 Analysis of Existing Chips

We apply the prime compiler retrospectively to five recent analog accelerators to assess whether their manual partitioning decisions were optimal.

Key finding: Every chip we analyzed left energy on the table by keeping nonlinear operations digital that could have been mapped to analog primes. The most common missed opportunity is GELU (P11·P6), which has been demonstrated on RRAM [15, 16] but was not adopted by any of the five chip teams. The second most common is normalization, which all five teams kept digital—the AGC-based RMSNorm described in §5 resolves this.

Table 6: Retrospective analysis: manual vs. prime-compiler partitioning of recent analog chips.

Chip	Analog / Digital	Compiler suggestion	Missed opportunity
IBM 14nm PCM [1]	MatMul analog; Softmax, Norm, GELU digital	Move GELU (P11-P6) and RMSNorm (AGC) to analog	~100% GELU + Norm energy savings
TSMC CIM [2]	MatMul analog; activations digital	Move activations analog; fuse Softmax+Loss (Race)	Activation energy + multi-comp 5×
Jülich [5]	QK^T analog; Softmax replaced by HardSigmoid	Use SMTJ Race for exact Softmax	Exact softmax + 5× multi-comp
Extropic XTR-0 [3]	Boltzmann machine analog; digital control	Extract partition fn. + entropy from BM relaxation	Class 2.4 multi-comp (3×
Normal CN101 [4]	RLC sampling analog; matrix ops digital	Extend to RM-SNorm via AGC feedback	On-chip normalization

7 Discussion

7.1 What the Prime Compiler Does Not Do

The compiler does not accelerate computation on *existing* digital hardware (CPUs, GPUs). The digital abstraction layer in modern processors seals the underlying transistor physics behind billions of logic gates. There is no software API to operate a GPU’s MOSFETs in subthreshold mode.

The compiler’s value is in *chip design*: systematically answering the analog/digital partitioning question that every mixed-signal accelerator team faces.

7.2 The Multi-Computation Advantage

Multi-computation fusion [7] provides 2–12× effective throughput multipliers by extracting additional observables from physical events that are already occurring. The prime compiler is the first tool to *automatically detect* these opportunities in arbitrary compute graphs.

Without the compiler, fusion opportunities are found (or missed) by manual inspection. With the compiler, every subgraph matching a known multi-computation class is flagged and quantified.

7.3 Towards Training

On analog-only hardware, training is blocked by X4 (symbolic tracing for backpropagation). On GPUs with analog extensions, X4 is available via `torch.compile`, enabling a hybrid approach: forward pass on analog (fast, low-energy), backward pass on digital (exact gradients).

The prime compiler supports this by partitioning the forward graph into analog and digital domains while ensuring that the backward graph (generated by X4) operates entirely in the digital domain with correct gradients.

8 Related Work

CIM compilers. CIM-MLC [27] provides a multi-level compiler for compute-in-memory accelerators (ASPLOS 2024), and CINM [28] generalizes across CIM and CNM paradigms via MLIR. CMSwitch [29] jointly optimizes compute-mode and memory-mode switching in CIM arrays. All three treat CIM arrays as “noisy MAC units” and route everything else to digital—none decompose operations into analog-native primitives or reason about *which* operations beyond MAC are physically suited to analog.

Hybrid analog-digital SoCs. DIANA [30] integrates analog CIM and digital accelerators on one SoC; its compiler HTVM [31] dispatches layers to either domain based on hardware capability matching, not operation semantics. The IBM mixed-precision supernetwork [32] uses NAS to find layer-to-hardware assignments—the closest existing work to systematic partitioning, but empirical (search-based) rather than principled (decomposition-based). A 2024 framework for DNN inference partitioning in hybrid analog-digital chiplets [33] operates at layer granularity with accuracy-vs-energy tradeoffs.

Analog compilers. Legno [22] compiles differential equations to reconfigurable analog devices, mapping physics to computation—the closest precedent to our approach. Hasler’s FPAA toolflow [23] programs floating-gate analog arrays from high-level descriptions. Both target analog-only execution of general computations; neither addresses the analog/digital partitioning problem for neural networks.

DNN compilers. TVM, XLA, and `torch.compile` partition across digital hardware types (CPU/GPU/TPU) but not across physical domains. NIR [34] defines a neuromorphic intermediate representation for spiking systems across 7 simulators and 4 hardware platforms, but targets spike-based computing, not analog/digital CIM partitioning.

Analog simulation. NeuroSim [20], CrossSim [21], and IBM’s AIHWKIT [35] simulate analog non-idealities for accuracy estimation. NavCim [36] performs design-space exploration for CIM architectures. None factorize operations into primes or detect multi-computation opportunities.

Normalization in analog. IMPACT [25] implements batch normalization as a linear gain+offset in the ADC—a pre-computed affine transform, not true normalization. The ReRAM ALN [26] normalizes in the charge domain with simplifications. No prior work implements full RMSNorm or LayerNorm in analog with the inverse-square-root computation.

Key gap. No existing framework operates at *operation granularity* with a *principled decomposition* into analog-native primitives. All partitioning decisions in the literature are either (a) layer-level, (b) hardware-capability-driven, or (c) empirical search. The prime compiler is the first to provide a theory-driven, operation-level partitioning framework grounded in physics.

9 Conclusion

We have introduced Computational Primes—12 irreducible operations with physical implementations and 4 without—as a systematic basis for decomposing neural network computation. The resulting prime compiler automates the analog/digital partitioning problem that every mixed-signal accelerator team currently solves by hand.

Key results:

- 107 algorithms factorized: 79 fully analog-mappable (73%), 22 GPU-mappable (21%), 6 unmappable (6%).
- RMSNorm mapped to analog for the first time via AGC feedback (~ 12 ns, no ADC).

- Retrospective analysis of 5 chips shows all left 15–40% energy savings uncaptured.
- Multi-computation fusion detected automatically, yielding 2–12 $\times$ throughput.

The prime compiler does not make existing GPUs faster. Its value is in chip design methodology: replacing ad hoc partitioning with a systematic, automatable framework. As analog accelerators move from research prototypes to production silicon, the need for such a framework will only grow.

Code and the full factorization table are available at: <https://github.com/Biech95/prime-compiler>.

References

- [1] A. Chen *et al.*, “Demonstration of transformer-based ALBERT model on a 14nm analog AI inference chip,” *Nature Communications*, 2025.
- [2] W.-S. Khwa *et al.*, “A mixed-precision memristor and SRAM compute-in-memory AI processor,” *Nature*, 2025.
- [3] A. Jelencic *et al.*, “An efficient probabilistic hardware architecture for diffusion-like models,” *arXiv:2510.23972*, 2025.
- [4] D. Melanson *et al.*, “Thermodynamic computing system for AI applications,” *Nature Communications*, vol. 16, 3757, 2025.
- [5] N. Leroux *et al.*, “Analog in-memory computing attention mechanism for fast and energy-efficient large language models,” *Nature Computational Science*, 2025.
- [6] M. Bieg, “Stochastic Barrier Attention: Normalization-free softmax sampling via thermodynamic race processes,” *Preprint*, 2025.
- [7] M. Bieg, “Multi-computation from exponential race events: Extracting simultaneous observables from stochastic barrier arrays,” *Preprint*, 2026.
- [8] M. Prezioso *et al.*, “Training and operation of an integrated neuromorphic network based on metal-oxide memristors,” *Nature*, vol. 521, pp. 61–64, 2015.
- [9] J. Sillman, “Analog softmax via subthreshold translinear loops,” *arXiv:2305.13649*, 2023.
- [10] M. K. Tageldeen *et al.*, “Learning in log-domain: Subthreshold analog AI accelerator,” *arXiv:2501.13181*, 2025.
- [11] M. Amin *et al.*, “MRAM-based analog sigmoid for in-memory computing,” *GLSVLSI*, 2022.
- [12] T. Dalgaty *et al.*, “DenRAM: Neuromorphic dendritic architecture with RRAM,” *Nature Communications*, 2024.
- [13] H. Kim *et al.*, “Cryptographic transistor for true random number generation,” *Science Advances*, vol. 10, eadk6042, 2024.
- [14] B. Gilbert, “Translinear circuits: A proposed classification,” *Electronics Letters*, vol. 11, no. 1, pp. 14–16, 1975.
- [15] H. Alahmadi *et al.*, “RRAM-based Swish activation function on 28nm FD-SOI,” *IEEE ICECS*, 2022.

- [16] H. Errahmouni Barkam *et al.*, “RACE-IT: Reconfigurable analog CAM-crossbar engine for in-memory transformer acceleration,” *arXiv:2312.06532*, 2023.
- [17] X. Zhang *et al.*, “Compute-in-memory implementation of state space models for event sequence processing,” *Nature Communications*, 2025.
- [18] B. Zhang and R. Sennrich, “Root mean square layer normalization,” *NeurIPS*, 2019.
- [19] M. Carandini and D. J. Heeger, “Normalization as a canonical neural computation,” *Nature Reviews Neuroscience*, vol. 13, pp. 51–62, 2012.
- [20] P.-Y. Chen *et al.*, “NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures,” *IEEE TCAD*, 2018.
- [21] S. Agarwal *et al.*, “CrossSim: Accuracy simulation of analog in-memory computing,” *Sandia National Lab*, 2023.
- [22] S. Achour *et al.*, “Legno: Noise-aware dynamical system compilation for analog devices,” *ASPLOS*, 2020.
- [23] J. Hasler, “Large-scale field-programmable analog arrays,” *IEEE Proceedings*, 2020.
- [24] R. Wei *et al.*, “LightMamba: Efficient Mamba acceleration on FPGA,” *DATE*, 2025.
- [25] A. Kneip *et al.*, “IMPACT: A 1-to-4b 813-TOPS/W 22nm FD-SOI CIM-SRAM with multi-bit analog batch-normalization,” *IEEE JSSC*, 2023.
- [26] Y.-C. Hou *et al.*, “A ReRAM-based CNN accelerator using the analog layer normalization technique,” *IEEE Trans. Electron Devices*, 2022.
- [27] Y. Zhao *et al.*, “CIM-MLC: A multi-level compilation stack for computing-in-memory accelerators,” *ASPLOS*, 2024.
- [28] S. Khan *et al.*, “CINM: A compilation infrastructure for heterogeneous compute in-memory and compute near-memory paradigms,” *ASPLOS*, 2024.
- [29] Z. Chen *et al.*, “Be CIM or Be Memory: A dual-mode-aware DNN compiler for CIM accelerators,” *ASPLOS*, 2025.
- [30] K. Ueyoshi *et al.*, “DIANA: An end-to-end energy-efficient digital and analog hybrid neural network SoC,” *ISSCC*, 2022.
- [31] N. Delm *et al.*, “HTVM: Efficient neural network deployment on heterogeneous TinyML platforms,” *DAC*, 2023.
- [32] H. Benmezziane *et al.*, “Supernet-based efficient mapping to mixed-precision hardware using model adaptation,” *Nature Communications*, 2026.
- [33] A. Varadarajan *et al.*, “Deep neural network inference partitioning in embedded hybrid analog-digital systems,” *IEEE*, 2024.
- [34] J. E. Pedersen *et al.*, “Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing,” *Nature Communications*, 2024.

- [35] M. J. Rasch *et al.*, “Using the IBM analog in-memory hardware acceleration kit,” *APL Machine Learning*, 2023.
- [36] S. Kim *et al.*, “NavCim: Comprehensive design space exploration for analog CIM architectures,” *PACT*, 2024.